ProveIt research engineering review

Eight additional ways to simplify the Lean proofs

DiophantineConsistency

New proposals relative to the incoming reports, with checked proof artifacts

Prepared for Vladimir Reshetnikov
26 October 2026

Reviewed commit: `e73fa2e916c9bb6f61c79e4fe2cd02b5f43e9d11`

Environment: Lean 4.32.0
Mathlib: `81a5d257c8e410db227a6665ed08f64fea08e997`.

Result. Six concrete refactorings provide exact patches with a combined reduction of **109 physical source lines across seven production files**. Two further proposals provide checked foundations for compositional primitive-recursiveness proofs and canonical carry normalization.

The strongest immediate change preserves the rational witnesses during descent to integers. The most reusable changes identify the exact graph of the tree decoder and use finiteness to recover predecessors of an invariant predicate.

The archive records which complete modules were rebuilt, which checks cover a submodule or declaration, and which integrations remain proposed.

Scope. These are proof refactorings and supporting lemmas. No new consistency theorem or smaller Diophantine equation is claimed. No repository-wide speedup is inferred from shorter source. The recommendations were screened against the current incoming archive and its explicitly referenced earlier proof-simplification packet.

Contents

1 Findings and recommended order 1

2 Snapshot, evidence, and the non-repetition check 2

3 Preserve rational roots when descending to integers 3

4 Decompose the product's units once 4

5 Describe the tree decoder by its exact graph 4

6 Use finiteness to recover an invariant predecessor 5

7 Prove the support envelope before individual coefficients 6

8 Rotation is convolution with a point mass 7

9 Automate extensional primitive-recursiveness composition 8

10 Canonical normalization from a finite maximum principle 9

11 Measurements, verification, and remaining integration work 10

12 Artifact guide 12

1 Findings and recommended order

The most useful additional simplifications come from retaining information that the current proofs discard, or proving precisely the property their clients need. This produces smaller proofs without changing the numerical encodings. Examples include retaining a rational root's sign while proving it integral, collecting two decoder invariants in one induction, and using surjectivity on a finite invariant subset instead of constructing an entire periodic orbit.

The review intentionally excludes the earlier reports' major themes: lookup and membership APIs, witness-coordinate transport, checked history folds, scanner simulations, power certificates, generic DAG invariants, sum-of-squares infrastructure, half-radix identities, shared field residues, source-register bundles, and model-completeness transport. The recommendations below use different mathematical mechanisms.

ID	Mechanism	Main target	Validation
1	Preserve rational witnesses during integer descent	NearSquareFinalizer; MarginFinalizer	Complete original and patched modules
2	Decompose product units once	ShortSystem	Complete patched module
3	One exact graph theorem for the tree decoder	AnnotatedProofCode.Tree	Complete affected Tree submodule
4	Reflect an invariant through a finite injection	NativeFieldCycles2011	Complete original and patched modules
5	Derive coefficient support from factor degrees	SelectiveCarry	Exact declaration and original encoding dependencies
6	Derive rotation from a point-mass convolution	NativeCyclicRotation	Complete original and patched modules
7	Register Primrec as a compositional function property	Computability	Generic constructor and client prototypes
8	Prove carry uniqueness by a finite maximum principle	NativeCyclicCarry; word-window layers	General uniqueness and packing prototypes

Items 1–6 are independent except that the two finalizer changes in item 1 should be applied together. The proposed bundle preserves all existing public declarations in the affected files. The private `unit_eq_one` helper becomes unnecessary and is removed. The two foundation proposals do not have production patches or claimed net production line savings.

The immediate adoption order is 1, 2, 4, 6, then 3 and 5 with their larger enclosing modules rebuilt. Items 7 and 8 merit separate integration changes so that the cost of their shared infrastructure remains visible. This ordering reflects completed validation and review effort rather than a claim about global performance.

2 Snapshot, evidence, and the non-repetition check

The active library at the reviewed commit contains **570 Lean files and 76,877 physical lines**. Every one of those 570 file contents was checked against its Git blob identifier from the pinned tree; there were zero mismatches. These are inventory figures, not counts of kernel proof steps. The package includes the file inventory, source-integrity result, pinned toolchain and manifest, and the original files touched by patches.

At this commit, `docs/incoming` contains exactly two entries: its README and `diophantine-lean-simplification-2026-10-26.zip`. The review covered the entire 267,590-byte, 1,952-line README, the complete 920-line manuscript in that archive, and the complete 442-line manuscript in the earlier packet at `research/lean-proof-simplification-20261026`, together with its three supplementary source notes. That earlier packet is explicitly linked by the incoming index. Its placement note links an `ADOPTION.md` file that is absent at this snapshot; no adoption status is inferred from that link.

The evidence is reproducible in `evidence/candidate-novelty-crosswalk.json`, `evidence/exclusion-catalog.json`, and `evidence/incoming-index-scan.json`. The exclusion catalog records 32 existing technique families and their source locations. The source index is available at the pinned incoming README; the older packet is at the pinned research directory.

New mechanism	Closest earlier theme	Substantive difference
Rational descent	Source/Pell fact bundles	Preserve the exact root by denominator descent; eliminate sign reconstruction.
Integer units	Shared algebraic infrastructure	A single unit decomposition replaces four separately manufactured complementary products.
Exact decoder graph	Scanner and history invariants	Characterize recursive decoding on arbitrary codes and exact fuel bounds.
Finite invariant reflection	Forward induction invariants	Obtain a predecessor using finiteness and injectivity.
Degree-first support	Weighted multivariate degree reuse	Reverse a univariate support proof: degree bounds imply coefficient vanishing.
Point-mass convolution	Half-radix and residue APIs	Derive translation semantics as a specialization of multiplication.
Primrec automation	PR recursor and scanner programs	Automate extensional function-property composition with existing closure theorems.
Carry uniqueness	Coherent window witnesses	Supply the missing maximum-principle uniqueness proof; coherence is only its application.

One plausible additional direction was deliberately rejected: upgrading aligned cells to a structured equivalence with $\mathbb{Z}/N\mathbb{Z}$. The earlier manuscript already suggests an aligned-cell equivalence to simplify reindexing. Strengthening it to an additive equivalence is too close to that proposal to count as an additional headline here.

"Additional" means new relative to this audited proposal set. The underlying mathematics and Mathlib APIs are established. This report makes no claim of global mathematical novelty or exhaustive reading of every historical construction paper linked transitively by the incoming index. The complete index's

summaries were scanned; the two proof-simplification packets and supplementary notes were read in full.

3 Preserve rational roots when descending to integers

3.1 The proof loses more information than necessary

The targets are `NearSquareFinalizer.lean` (lines 94–167) and `MarginFinalizer.lean` (lines 20–77). The norm argument first obtains rational roots r, s with

$$r^2 = a, \qquad s^2 = b, \qquad \theta/\omega = r + ws.$$

The current integer descent converts the first two equalities into existential squarehood statements, obtains fresh integer roots, and then reconstructs the signs. There are four combinations to handle because each replacement root may differ by a sign from the rational root used in the linear identity.

There is a stronger route: prove that the *already chosen rational number* is an integer. Its sign and every equation involving it then survive automatically.

3.2 The general denominator lemma

If $q \in \mathbb{Q}$, $n > 0$, and $q^n \in \mathbb{Z}$, then the reduced denominator of q is one. Mathlib's canonical rational representation already supplies the required denominator identity. The checked proof is:

```
theorem num_cast_of_pow_eq_int {q : ℚ} {a : ℤ} {n : ℕ}
    (hn : n ≠ 0) (h : q ^ n = (a : ℚ)) : (q.num : ℚ) = q := by
  apply Rat.coe_int_num_of_den_eq_one
  apply (pow_eq_one_iff_left hn).mp
  simpa only [Rat.den_pow, Rat.den_intCast] using congrArg Rat.den h
```

Taking denominators yields $\mathrm{den}(q)^n = 1$; the nonzero exponent forces $\mathrm{den}(q) = 1$. The existing theorem identifying the integer numerator with a denominator-one rational finishes the proof. No choice of a new square root occurs. See `Mathlib/Data/Rat/Defs.lean` (the numerator identity) and Lean 4.32.0, Init/Data/Rat/Lemmas.lean for `Rat.den_pow`.

The condition $n \neq 0$ matters: every rational has zeroth power one. The proof does not require positivity of q, and it therefore retains the negative-root cases without splitting on them.

3.3 Descent of the complete witness tuple

Apply the lemma to both square equations. Rewrite the rational identity using the casts of r.num and s.num, clear the nonzero denominator ω, and use `exact_mod_cast`. The integer witnesses are precisely those two numerators. The three required integer equations follow from the three rational equations with their original signs intact.

The implementation also puts the more general separated-root theorem in `NearSquareFinalizer`, where both finalizers can use it. The existing stronger-bound theorem becomes its corollary; `MarginFinalizer` preserves its public theorem as a compatibility wrapper. This movement is secondary to the mathematical simplification: it prevents the same witness-preserving descent from being copied twice.

The two files together shrink from **432 to 390 lines**. The first file remains 255 lines because it takes ownership of the shared general result; the second shrinks from 177 to 135. This accounting includes the new helper and compatibility declarations. The original and patched full modules compile, and the audited exported theorem axiom sets agree.

Adoption boundary. Keep the original hypotheses that produce rational roots and exclude the degenerate norm cases. The denominator argument simplifies the descent step; it does not discharge those hypotheses. Its benefit extends to any later proof that has already chosen rational algebraic witnesses and needs to preserve additional relations among them.

4 Decompose the product's units once

The target `ShortSystem.lean` (lines 20–61) proves an equivalence from

$$n_0 n_1 n_a n_k n_t n_s = 1$$

under explicit assumptions excluding the negative unit for selected factors. The current proof proves four separate multiplication-equals-one facts, rearranging the other factors with `ring`, then invokes a helper on each.

A product equal to one is a unit. In the commutative integer ring, `IsUnit.mul_iff` recursively gives that every factor is a unit. Finally, `Int.isUnit_iff` says an integer unit is either 1 or -1. One simplification pass extracts the entire tuple of alternatives:

```
have hu : IsUnit (n0 * n1 * na * nk * nt * ns) :=
  hp.symm ▸ isUnit_one
simp only [IsUnit.mul_iff, Int.isUnit_iff, or_iff_left h0,
  or_iff_left h1, or_iff_left hs] at hu
```

The proof then obtains $n_s = 1$, uses the existing conditional hypothesis $n_s = 1 \Rightarrow n_a \neq -1$, and resolves the remaining sign for n_a. Substitution into the original equation gives $n_k n_t = 1$.

That last product is deliberately retained: the hypotheses do not force $n_k = n_t = 1$, since both could equal -1. The patch preserves this distinction and does not strengthen the theorem accidentally. The relevant API is `Mathlib/Algebra/Group/Int/Units.lean` (`Int.isUnit_iff`) and the multiplicative-unit lemmas it imports.

The complete `ShortSystem` file shrinks from **1,117 to 1,104 lines**, including removal of its unused private helper. The full patched module compiles. This is a small, direct simplification with no new production helper library.

5 Describe the tree decoder by its exact graph

5.1 Fuse two parallel soundness inductions

In `AnnotatedProofCode.lean` (lines 78–155), `decoded_depth_le` and `quote_of_decode` traverse the same fuel and code cases. One retains a depth bound, the other a quotation equality. The shared induction invariant should retain both:

$$\text{decode}(f, c) = \text{some}(t) \implies \text{quote}(t) = c \ \wedge \ \text{depth}(t) \leq f.$$

Neither projection is harder to prove in the combined induction. At a fork, both child calls succeed, both quotation equalities reconstruct the original pair code, and both depth bounds imply the maximum-depth bound.

A further local improvement removes explicit failure branches. After exposing the nested `Option.bind` expression, rewrite with `Option.bind_eq_some_iff`. The success hypothesis itself supplies successful child values:

```
simp only [Option.bind_eq_some_iff, Option.some.injEq] at h
obtain ⟨a, ha, b, hb, rfl⟩ := h
obtain ⟨hqa, hda⟩ := ih ha
obtain ⟨hqb, hdb⟩ := ih hb
```

The proof never needs to inspect impossible `none` results separately. The existing `Nat.pair_unpair` equality and the depth formula finish the two conjuncts.

5.2 A specification on arbitrary codes

Combining the soundness product with the unchanged `decode_quote` completeness theorem gives:

```
decode fuel code = some t ↔ t.quote = code ∧ t.depth ≤ fuel
```

This is stronger than a theorem restricted to quoted inputs. The input `code` is an arbitrary natural number, including malformed codes. The equivalence does not assert that every number has a parse; it characterizes exactly when a particular tree is returned. It also states the exact fuel threshold, not merely that sufficiently large fuel works.

The old API remains as projections and corollaries. The existing quoted-input success equivalence is shortened using the exact graph and quotation injectivity. Fuel monotonicity becomes a further optional corollary, with no new recursion proof.

The patch reduces the whole file from **461 to 439 lines**; changed declaration blocks reduce from 66 to 44. Both totals include the combined soundness theorem, the exact graph theorem, and all retained compatibility declarations. Executable definitions are unchanged.

Validation boundary. The complete original `Tree` submodule with the actual patch applied, including every original Tree theorem, compiles under Lean 4.32.0. The enclosing proof-checker module's full import closure was not rebuilt. The archive includes a complete proposed replacement file and a precise patch so that this remaining integration check is straightforward.

6 Use finiteness to recover an invariant predecessor

6.1 The generic mathematical principle

Let A be finite, $f : A \to A$ injective, and P a forward-invariant predicate:

$$P(x) \Longrightarrow P(f(x)).$$

The restriction of f to $\{x : P(x)\}$ is an injection from a finite set to itself, hence is surjective. Therefore $P(f(x)) \Rightarrow P(x)$. More usefully, every point satisfying P has a predecessor satisfying P.

Mathlib already packages the required result in `Set.Finite.injOn_iff_bijOn_of_mapsTo`; see `Mathlib/Data/Set/Finite/Basic.lean` (restricted injections and bijections). The production patch uses this theorem directly, so no extra generic helper has to be added to the library.

6.2 Application to the offset-19 exclusion

The target is `NativeFieldCycles2011.lean` (lines 65–91). After the existing argument identifies the exceptional source value as 511, put

$$P(i) \;:\Longleftrightarrow\; S(i) \equiv 0 \pmod 2.$$

The existing `successor_mod_four` implies forward invariance of P. The proposed proof restricts the given finite permutation to the even-value subset and obtains a predecessor j of the allegedly 510-valued point i, with $S(j)$ even. Applying `successor_mod_four` to j yields $S(i) \equiv 0 \pmod 4$, contradicting $510 \equiv 2 \pmod 4$.

The current proof obtains the same predecessor indirectly: it proves evenness along all iterates, finds a positive period, constructs the iterate at period minus one, and proves that one successor returns to the starting point. None of that orbit arithmetic is required once restricted surjectivity is available.

The complete file shrinks from **111 to 104 lines**. Its periodic-points import is replaced by the finite-set API import. Both complete versions compile, public theorem statements agree, and the audited axiom sets agree.

Sharp limits. The finite and injective hypotheses are essential to the generic reflection principle. Successor on the infinite natural numbers preserves positivity but does not reflect it at zero. A noninjective map on a finite set can enter an invariant subset from outside. The patch retains all needed assumptions. It also leaves the legitimate 511 self-loop and the separate offset-20 theorem unchanged; the latter still needs no finiteness or permutation hypothesis.

7 Prove the support envelope before individual coefficients

The target `SelectiveCarry.lean` (lines 26–79) concerns

$$U(X) = \sum_i u_i X^{w_i}, \qquad P(X) = U(X)^2 V(X).$$

The hypotheses bound every w_i by W and every complementary exponent in V by D. The file already proves $\mathrm{natDegree}(V) \le D$. Nevertheless, the current coefficient-vanishing proof expands the full product into four nested finite sums and proves each out-of-range monomial coefficient zero.

The shorter route uses only

$$\mathrm{natDegree}(U) \le W, \qquad \mathrm{natDegree}(U^2 V) \le 2W + D.$$

If $D + 2W < n$, the coefficient at n is zero by the existing coefficient/degree theorem. The proposed proof uses `natDegree_sum_le_of_forall_le`, `natDegree_monomial_le`, `natDegree_pow_le_of_le`, `natDegree_mul_le_of_le`, and `coeff_eq_zero_of_natDegree_lt`. These are the standard upper-bound APIs from `Mathlib/Algebra/Polynomial/BigOperators.lean` (degree bounds for polynomial sums) and the polynomial degree library.

```
have he : (encoded w u).natDegree ≤ W := by
  unfold encoded
  exact natDegree_sum_le_of_forall_le _ _
    (fun i _ => (natDegree_monomial_le (u i)).trans (hw i))
have hd := natDegree_mul_le_of_le (natDegree_pow_le_of_le 2 he)
  (combined_degree_le w s a E D hD)
apply coeff_eq_zero_of_natDegree_lt
change (encoded w u ^ 2 * combined w s a E).natDegree < n
omega
```

This reverses the organization of the support argument. It does not reuse weighted multivariate degree certificates, as the previous reports suggested elsewhere. It also does not introduce a circular dependence on `rowProduct_degree_lt`: the envelope is derived independently from the factors, using the already proved bound on `combined`.

Only upper bounds are needed. Exact degree equalities would demand extra nonvanishing hypotheses and would complicate zero polynomials or cancellation. The proposed inequalities remain valid in those cases. Natural truncated subtraction in the complementary exponent remains exactly as defined and bounded in the original hypotheses.

The exact replacement theorem compiles against the actual original encoding modules. Its proof body shrinks from 18 to 9 lines; the proposed whole file shrinks from **204 to 196 lines**. The patch adds an explicit `Mathlib.Algebra.Polynomial.BigOperators` import, included in that net count, rather than relying on a transitive import. The full downstream packing module was not rebuilt. The related factor-degree helper already present in `SplitSourceBounds` suggests further clients, but their savings are not counted here.

8 Rotation is convolution with a point mass

The target is `NativeCyclicRotation.lean` (lines 18–39). Its `word_rotate` proof reindexes a finite sum, transports modular exponent identities through the sum, and factors a radix power. The preceding `NativeCyclicConvolution.lean` (lines 55–85) theorem already proves the more general multiplication/convolution congruence.

For a cyclic index t, let $\delta_t(i) = 1$ when $i = t$ and zero otherwise. Then

$$(\delta_t * c)(k) = \sum_i \delta_t(i) c(k-i) = c(k-t), \qquad \mathrm{word}_\rho(\delta_t) = \rho^{\mathrm{val}(t)}.$$

Substituting δ_t into the existing convolution theorem gives the rotation theorem immediately. Lean's `Pi.singlet1` is the needed point mass.

The checked production replacement needs no new declarations:

```
by
  classical
  simpa [word, convolution, rotate, Pi.single_apply, ite_mul] using
    word_convolution hp (Pi.single t 1) c
```

The archive additionally contains named point-mass identities as an explanatory prototype, but they are not required or charged as production helpers. This distinction matters when counting the patch's size.

The whole rotation file shrinks from **176 to 159 lines**. Both complete original and modified modules compile, and `word_rotate`'s axiom set is unchanged. The proof retains `NeZeron` and $1 \leq \rho$. It handles a one-position cycle through the general theorem and does not assume that the representative of the cyclic element one is the natural number one. No digit-size or no-carry premise is added.

This is a particularly low-cost instance of a useful search rule: inspect whether a hand-proved transformation is the action of a single basis element in a more general operation that is already formalized. Here the general theorem exists one import earlier, so the specialization adds essentially no architectural cost.

9 Automate extensional primitive-recursiveness composition

9.1 The target is the proposition about a function

In `Computability.lean` (lines 92–166), proofs of the formula constructors' primitive recursiveness manually repeat the expressions' syntax with nested `.comp` and `.pair` applications. The definitions already specify that composition tree. Mathlib's `fun_prop` tactic can construct the proof if `Primrec` and appropriate closure rules are registered as a function property.

The proposal is separate from the earlier reports' PR program optimizers, recursor congruence, and scanner simulation lemmas. It acts on extensional `Primrec` propositions. The executable programs, codes, and representation-size theorems stay governed by their existing proofs.

The prototype registers the property locally, together with identity, constants, composition, projections, pairing, arithmetic adapters, and the six formula-constructor adapters. A client becomes:

```
theorem clos_rule ... :
    Primrec (fun v => clos P (a v) (b v) (c v) (d v)) := by
  unfold clos
  fun_prop (disch := assumption)
```

The displayed ellipsis suppresses the parameters and hypotheses for readability; complete executable declarations are in `lean/PrimrecFunProp.lean`. For the registration contract see `Mathlib/Tactic/FunProp.lean` (user-defined function properties), and for the available closure theorems see `Mathlib/Computability/Primrec/Basic.lean` (`Primrec` and $\texttt{Primrec}_2$).

9.2 Two interface details are essential

First, the project's pairing is kept as a parameter `P`, with premise $\texttt{Primrec}_2\texttt{P}$. That is the fact the upstream `pr_P` already provides. The prototype does not substitute a different library pairing merely because its primitive-recursiveness theorem is convenient.

Second, the registered conclusion shape must be `Primrec`. Simply assigning the attribute to a theorem whose declared head is $\texttt{Primrec}_2$ did not solve the tested `Primrec` goals. The checked arithmetic rules explicitly produce `Primrec(funv=>...)`. For the nested axiom stack, expose the local pairing hypothesis in uncurried form:

```
change Primrec (fun p : Nat × Nat => P p.1 p.2) at hP
unfold axStack
fun_prop (disch := assumption)
```

The prototype compiles twelve clients: rho1, rho2, rho3, rho4, rho5, rho6, rho8, growG, req, clos, axStack, and a nested product-input example. There is no rho7 claim: the artifact follows the actual constructor family used in the source.

9.3 Costs and adoption

The tested clos and rho2 proof bodies use two lines each; the axiom-stack body uses three, including the local conversion. The corresponding upstream bodies have five, five, and nine lines. These are client measurements only. Registration and adapter declarations cost source lines, so a whole-module net saving and speedup have not been measured or claimed.

A suitable first integration is a small local rule module and migration of the eleven constructor proofs. Keep registrations scoped while validating rule search and error messages. Leave recursive-program construction and realization proofs explicit. The criterion for success is that changing a constructor expression also updates its composition proof automatically, without a second manually synchronized expression tree.

10 Canonical normalization from a finite maximum principle

10.1 What must be fixed to obtain uniqueness

The existing `NativeCyclicCarry.lean` (lines 24–75) construction finds bounded carries by a monotone fixed point. Its documentation correctly warns that quotient recurrence alone need not have a unique solution. The new result fixes the raw vector r and the normalized digit vector d, then proves uniqueness of the carries satisfying

$$r_i + k_{\mathrm{prev}(i)} = d_i + \rho k_i.$$

The theorem works on any finite index type and any predecessor function when $\rho > 1$. It does not require the predecessor to be injective or cyclic, and it does not require an a priori carry bound.

Suppose k and ℓ are two solutions and set $\Delta_i = k_i - \ell_i \in \mathbb{Z}$. Subtracting their equations gives

$$\Delta_{\mathrm{prev}(i)} = \rho \Delta_i.$$

Choose i where Δ_i is maximal. Then

$$\rho \Delta_i = \Delta_{\mathrm{prev}(i)} \le \Delta_i.$$

Because $\rho > 1$, this forces $\Delta_i \le 0$. Every difference is therefore nonpositive. Interchanging the two solutions proves the reverse inequality, hence equality pointwise. On an empty index type, equality is vacuous; the Lean proof selects a maximum only after introducing an actual index.

The checked file `CanonicalCarry.lean` formalizes this argument through `Finset.exists_max_image`, integer casts of the two local equations, and a short nonlinear arithmetic step. The theorem `carries_unique_of_same_digits` follows by antisymmetry. See `Mathlib/Data/Finset/Max.lean` (`exists_max_image`).

10.2 A complete packed word fixes the digit vector

The second checked component uses the existing theorem `Nat.ofDigits_inj_of_len_eq`. For fixed length n, radix $\rho > 1$, and digits strictly below ρ, equality

$$\mathrm{ofDigits}_\rho(a) = \mathrm{ofDigits}_\rho(b)$$

implies $a = b$. Leading zero digits do not cause an ambiguity because the lengths agree. The proof transfers list equality back to `Finn→Nat` by `List.ofFn_injective`; see `Mathlib/Data/Nat/Digits/Defs.lean` (fixed-length digit injectivity).

Combining this with the maximum principle gives a checked `normalization_unique` theorem: equal full packed words and equal raw data determine both digits and carries, for the same radix, length, and predecessor function. The cyclic source already has `word_eq_ofDigits`, so the bridge from its word representation to this theorem is explicit and existing.

10.3 The exact failure mode to avoid

Equality of full words cannot be replaced by congruence modulo $\rho^n - 1$. For one position, radix two, and raw digit one, both

$$(k, d) = (0, 1), \qquad (k, d) = (1, 0)$$

satisfy the local decomposition and quotient recurrence. Their packed words are one and zero; these are congruent modulo $2^1 - 1 = 1$. Thus neither recurrence alone nor modular word equality selects a unique normalization.

The source's `NativeCyclicCarryWord.lean` (lines 63–103) already identifies the normalized word with a *positive strict representative* F, ruling out the exceptional all-maximum-digit representative. Once both normalizations have that same complete word F, digit injectivity and carry uniqueness apply. This existing strict-word step is indispensable to the proposed integration.

10.4 How it can simplify later proof structure

The immediate candidate clients are `CurrentPolynomialWindow2011.lean` (lines 27–73) and its subsequent window-strengthening theorems. Instead of repeatedly maintaining the provenance of existential digit/carry witnesses, a shared uniqueness lemma would let two independently established properties be transported onto the same normalization. The carry vector could also be chosen canonically from the existing existence theorem, with all other normalizations proved equal to it.

That broader canonical interface is a proposal, not a measured patch. The prior supplementary notes already called for coherent low/high windows and mentioned a possible uniqueness argument. The additional contribution here is the actual checked maximum-principle proof and fixed-length packing result, together with their precise hypotheses. Rebranding the earlier coherence recommendation would not meet the non-repetition requirement.

11 Measurements, verification, and remaining integration work

11.1 Concrete source reductions

All counts below refer to complete proposed production files, including the helper overhead inside the patches. Separate explanatory examples are not counted as production code.

Production file	Original	Proposed	Removed
NearSquareFinalizer	255	255	0
MarginFinalizer	177	135	42
ShortSystem	1,117	1,104	13
AnnotatedProofCode	461	439	22
NativeFieldCycles2011	111	104	7
SelectiveCarry	204	196	8
NativeCyclicRotation	176	159	17
Total	2,501	2,392	109

This is about a 4.4% source reduction across the seven selected files. It is not a prediction for the whole project. In particular, some files contain long unchanged definitions or generated tables; physical line counts intentionally do not pretend to measure mathematical difficulty, proof-term size, or elaboration cost.

11.2 Validation levels

The finalizer pair, `ShortSystem`, `NativeFieldCycles2011`, and `NativeCyclicRotation` were checked as complete patched modules. Baseline comparisons were also run for the finalizer pair, the field-cycle module, and the rotation module. The decoder check covers its complete affected Tree submodule with all original Tree theorems retained. The coefficient-support check covers the exact replacement declaration against actual original encoding dependencies. The automation and normalization results are standalone prototypes.

The audited axiom outputs contain only ordinary Lean logical axioms: `propext`, `Classical.choice`, and `Quot.sound`, with some declarations using fewer. No delivered checked result contains `sorryAx`. The patch proposals do not add axioms, assumptions of the final theorem, or `native_decide`. Some unrelated original source proofs use their existing tactics; the refactorings preserve those parts.

No whole-project downstream build was performed. Passing a declaration or submodule check is not presented as passing the entire enclosing project. The logs and machine-readable validation records make these boundaries explicit. Timing records are replay observations in a shared environment, not controlled benchmarks; no speedup is asserted.

11.3 Review and replay plan

The archive contains a standalone Lake project for the eight mechanisms and three unchanged upstream support modules required by the support-envelope example. It pins Lean and Mathlib. `scripts/check_patches.py` checks the original hashes and runs a dry-run application of the complete patch bundle against a supplied ProveIt checkout. `scripts/replay.sh` builds the examples after dependencies are available. The README gives the exact commands.

For production adoption, apply the six refactorings to an isolated checkout at the pinned commit. Rebuild the seven affected modules, then the downstream library target. The remaining full-module checks for the decoder and coefficient-support patch are the next verification steps. Preserve the original public declarations and compare downstream axiom reports where those declarations are trust-boundary endpoints.

The `Primrec` automation should be integrated separately and measured including its rule library. The carry uniqueness theorem should first be connected to `word_eq_ofDigits` and the strict representative

theorem; only then should a canonical window interface be introduced. These steps keep the changes reviewable and avoid attributing a line saving to infrastructure whose clients have not yet migrated.

12 Artifact guide

Path	Contents
`report.tex`, `report.pdf`	This report's complete LaTeX source and compiled PDF.
`lean/`	Checked examples for all eight mechanisms; aggregate replay module; three unchanged upstream support modules.
`patches/`	Individual exact patches and one combined patch for all seven affected production files.
`proposed/`	Complete proposed replacement files for review.
`originals/`	Original versions of the seven touched files at the pinned snapshot.
`evidence/`	Validation logs, source and theorem metrics, source hashes, incoming-proposal exclusion catalog, candidate novelty crosswalk, and the full-index scan record.
`notes/`	Additional detailed proof and integration notes from the arithmetic, checker, and exclusion reviews.
`scripts/`	Patch applicability check, example replay, report rebuild, and inventory generation.
`SHA256SUMS.txt`	Checksums for the delivered package members.

The evidence separates three questions: whether a technique was already proposed, whether a replacement proves the stated theorem, and whether it simplifies the integrated development. The first two have direct artifacts here. The final question has concrete source measurements for six refactorings and an explicit integration plan for the remaining two.